Full-Stack Learning Roadmap

Spring Boot · MongoDB · Vue.js + Vite · Pinia · Axios

Spring Boot	MongoDB	Vue.js + Vite	Pinia	Axios

Build a complete task manager from scratch. Follow each phase in order and you will have a working RESTful API connected to a Vue.js frontend by the end. 5 phases · ~5 weeks.

Course Overview

Phase	Topic	Duration	Key Tech
1	**Backend Foundation**	Week 1	*Spring Boot, MongoDB, Lombok*
2	**RESTful API — Controllers**	Week 2	*@RestController, CORS*
3	**Vue.js + Vite Frontend**	Week 3	*Vue 3, Vite, Pinia*
4	**Axios — API Integration**	Week 4	*Axios, async/await*
5	**Integration & Deploy**	Week 5	*Postman, Railway, Vercel*

1 Backend Foundation — Spring Boot + MongoDB

Week 1

Architecture overview

Every HTTP request travels through four layers before hitting the database:

Client (Vue / Postman) → **Controller** (REST layer) → **Service** (Business logic) → **Repository** (Data access) → **MongoDB** (Atlas Cloud)

Setup steps

- Create a new project at start.spring.io — select Spring Web, Spring Data MongoDB, and Lombok dependencies.
- Configure application.properties with your MongoDB Atlas URI: spring.data.mongodb.uri=mongodb+srv://...
- Define the Task model — annotate with @Document and @Id.
- Create TaskRepository extending MongoRepository<Task, String> — zero boilerplate needed.
- Create TaskService injected with the repository — all business rules live here.

Task entity

Model — Task.java	Enum values
id **title** **description** priority → Enum (LOW / MEDIUM / HIGH) status → Enum (TODO / IN_PROGRESS / DONE)	**Priority enum:** LOW | MEDIUM | HIGH **Status enum:** TODO | IN_PROGRESS | DONE

Task.java — annotated model

```
@Document(collection = "tasks")
@Data  @NoArgsConstructor  @AllArgsConstructor
public class Task {
  @Id
  private String id;
  private String title;
  private String description;
  private Priority priority;
  private Status   status;
}
```

Lombok annotations (@Data, @NoArgsConstructor) auto-generate getters, setters, and constructors at compile time — no manual boilerplate required.

Key annotations to learn

Annotation	Purpose
`@Document`	Marks a class as a MongoDB collection document
`@Id`	Designates the MongoDB _id field
`@Service`	Marks a class as a Spring service bean
`@Autowired`	Injects a dependency automatically
`@Data (Lombok)`	Generates getters, setters, equals, hashCode, toString

2 RESTful API — Controller & Endpoints

Week 2

TaskBuddy API endpoints

Method	Endpoint	Description
GET	/api/tasks	Get all tasks
GET	/api/tasks/{id}	Get a single task
POST	/api/tasks	Create a new task
PUT	/api/tasks/{id}	Update a task
DELETE	/api/tasks/{id}	Delete a task

TaskController.java

```
@RestController
@RequestMapping("/api/tasks")
@CrossOrigin(origins = "http://localhost:5173") // Vue dev server
public class TaskController {

  @Autowired private TaskService service;

  @GetMapping
  public List<Task> getAll() { return service.findAll(); }

  @PostMapping
  public ResponseEntity<Task> create(@RequestBody Task t) {
    return ResponseEntity.status(201).body(service.save(t));
  }

  @PutMapping("/{id}")
  public Task update(@PathVariable String id, @RequestBody Task t) {
    return service.update(id, t);
  }

  @DeleteMapping("/{id}")
  public ResponseEntity<Void> delete(@PathVariable String id) {
    service.delete(id); return ResponseEntity.noContent().build();
  }
}
```

Always enable CORS with @CrossOrigin on your controller, or configure it globally via WebMvcConfigurer. Without it your Vue app will be blocked by the browser security policy.

HTTP status codes students must know

Code	Status	When to use
200	**OK**	Successful GET or PUT
201	**Created**	Successful POST — resource created
204	**No Content**	Successful DELETE — no body returned
400	**Bad Request**	Invalid payload or missing required fields
404	**Not Found**	Task ID does not exist in MongoDB

3 Vue.js + Vite Frontend Setup

Week 3

Scaffold the project

- **Run: npm create vite@latest taskbuddy-frontend -- --template vue**
- cd taskbuddy-frontend && npm install
- Install dependencies: npm install axios pinia vue-router
- Register Pinia and Vue Router in main.js — they must be added before mounting the app.
- Create a task store in src/stores/taskStore.js using defineStore.
- Build three components: TaskList, TaskCard, TaskForm. Connect them to the store.

Recommended folder structure

```
src/
├── api/
│   └── taskApi.js        // axios calls
├── stores/
│   └── taskStore.js      // pinia store
├── components/
│   ├── TaskList.vue
│   ├── TaskCard.vue
│   └── TaskForm.vue
├── views/
│   └── HomeView.vue
└── main.js
```

Pinia store — taskStore.js

```
import { defineStore } from 'pinia'
import * as api from '../api/taskApi'

export const useTaskStore = defineStore('tasks', {
  state: () => ({ tasks: [], loading: false }),
  actions: {
    async fetchAll() {
      this.loading = true
      this.tasks = await api.getAllTasks()
      this.loading = false
    },
    async add(task)  { this.tasks.push(await api.createTask(task)) },
    async remove(id) {
      await api.deleteTask(id)
      this.tasks = this.tasks.filter(t => t.id !== id)
    }
  }
```

```
})
```

Pinia is Vuex's modern replacement. It uses a flat structure — no mutations needed. The store's state is reactive; components that read it automatically re-render when data changes.

Pinia concepts to teach

Concept	Explanation
`defineStore`	Registers a named store; returns a composable hook
`state()`	Returns a reactive object — your app's data source of truth
`actions`	Functions that mutate state; can be async (API calls go here)
`getters`	Computed properties derived from state (e.g. filter by priority)
`storeToRefs()`	Destructures reactive refs from store without losing reactivity

4 Axios — Connecting Frontend to Backend

Week 4

Request flow

Vue Component → **Pinia** Action → **taskApi.js** Axios → **Spring** Controller → **MongoDB**

taskApi.js — all API calls in one place

```
import axios from 'axios'

const BASE = 'http://localhost:8080/api/tasks'

export const getAllTasks  = () => axios.get(BASE).then(r => r.data)
export const getTask     = id => axios.get(`${BASE}/${id}`).then(r => r.data)
export const createTask  = task => axios.post(BASE, task).then(r => r.data)
export const updateTask  = (id, task) =>
  axios.put(`${BASE}/${id}`, task).then(r => r.data)
export const deleteTask  = id => axios.delete(`${BASE}/${id}`)
```

Using the store in a Vue component

```
// Inside a Vue component <script setup>
import { onMounted } from 'vue'
import { useTaskStore } from '../stores/taskStore'
import { storeToRefs }  from 'pinia'

const store = useTaskStore()
const { tasks, loading } = storeToRefs(store)

onMounted(() => store.fetchAll())
// tasks is now reactive — use {{ tasks }} in the template
```

Error handling — always use try/catch

```
async fetchAll() {
  try {
    this.loading = true
    this.tasks = await getAllTasks()
  } catch (err) {
    this.error = err.response?.data?.message
      ?? 'Failed to load tasks'
  } finally {
    this.loading = false
```

```
    }
  }
```

> ⚠ err.response exists only for HTTP errors (4xx, 5xx). Network failures set err.request instead. Always check both when debugging connection issues between Vue and Spring Boot.

5 Full Integration — Test, Polish & Deploy

Week 5

Integration checklist

- Test all five endpoints with Postman before touching the frontend — confirm correct status codes and response shapes.
- Add filter by status — a Pinia getter: getters: { byStatus: state => s => state.tasks.filter(t => t.status === s) }
- Add exception handling in Spring: create a @ControllerAdvice class that returns meaningful JSON error responses.
- Add a loading spinner bound to store.loading and an error message bound to store.error in the Vue template.
- Deploy the backend to Railway or Render and the frontend to Vercel or Netlify. Update the Axios base URL accordingly.

Deployment summary

Test APIs	Error states	Deploy
Postman collections	Loading + 404 UI	Railway + Vercel

At this point students have a full-stack CRUD app: REST API → MongoDB → Axios → Pinia → Vue. The pattern applies to any future project — swap the entity, keep the structure.

Key Principles to Reinforce Every Week

→ Separation of concerns Controller / Service / Repository layers must never cross-contaminate each other's responsibilities.	**→ REST semantics** Use the right HTTP verb and the right status code — GET does not create, POST does not delete.
→ Reactive state Pinia owns the data; components only display it. Never fetch directly inside a component.	**→ Single source of truth** Never duplicate task data in multiple component states — the store is the one truth.
→ Always handle errors Wrap every async call in try/catch. Handle 4xx, 5xx, and network failures separately.	**→ Test backend first** Verify all endpoints in Postman before writing any frontend code.